

Sparse MoE GPT with Causal Cache and Backoff Language Modelling

DASE7506 Machine Learning — Mini-Project 1

FIELD	VALUE
Student ID	3036748359
Name	wang Nailin
Code commit	37f0bacc659ab609f5a0af5d153bdc92e2adb225
Code repository	https://github.com/wnailinx/MiniProject1
Checkpoint	https://github.com/wnailinx/MiniProject1/releases/download/ckpt/checkpoint.pt
Checkpoint SHA256	3e29029d5760d84e5a062bd6acddd57e1342a83844c15866d5d0e09b5c4c8100
Full-test CPU FP32 BPB	1.482848718

Abstract

This project trains a causal language model from random initialization on the supplied WikiText-2 training split. The final system uses a seven-layer GPT with RMSNorm, rotary position embeddings, SwiGLU and four top-1 routed experts in every block. Training adds two future-token prediction losses and selects an exponential moving average checkpoint. At inference time, the neural distribution is combined with an exact causal suffix cache and a compact absolute-discount 2- to 5-gram model built only from the training split. The final predictor obtains **1.460262 validation bits per byte (BPB)**. It uses 59.33 MiB of inference assets, 1.95 GiB peak CPU memory and 4.60 times the paired baseline CPU scoring time, satisfying the project limits.

1. Task and evaluation

The task is next-token prediction with the fixed BPE-2048 tokenizer. Evaluation uses independent causal windows of 256 tokens and reports

$$\text{BPB} = \frac{-\sum_t \log_2 p(x_{t+1} \mid x_{\leq t})}{\text{number of raw UTF-8 bytes}}.$$

Development and checkpoint selection use the validation split. The test split is evaluated only after freezing the method. The data, tokenizer and evaluator are unchanged. The final predictor must remain within five times the baseline CPU scoring time, 4 GiB peak evaluation RAM and 64 MiB of uncompressed inference assets.

2. Final method

2.1 Modern GPT backbone

The neural model has seven blocks, width 224, seven attention heads, context length 256 and dropout 0.1. It contains 13,263,264 parameters after adding the experts and multi-token heads. Input embeddings and output projection weights are tied.

The baseline LayerNorm, learned absolute positions and GELU feed-forward network are replaced by:

- **RMSNorm**, which normalizes by root mean square without subtracting the mean;
- **RoPE**, which rotates query and key pairs using position-dependent angles;
- **SwiGLU**, which computes

$$\text{SwiGLU}(x) = W_o [\text{SiLU}(W_g x) \odot W_v x].$$

The SwiGLU hidden width is approximately $(8d/3)$, giving a parameter cost close to the baseline **4d** GELU feed-forward layer. Attention remains strictly causal. Residual output matrices use GPT-2-style scaled initialization with standard deviation $(0.02/\sqrt{2L})$.

2.2 Sparse mixture of experts

Every block contains four independent SwiGLU experts. A learned router calculates

$$p(e \mid x) = \text{softmax}(W_r x)$$

and sends each token to the highest-probability expert. All tokens use the shared attention layer, but each token executes only one feed-forward expert. This increases conditional parameter capacity without evaluating all four experts for every token.

The selected router probability is divided by a detached copy of itself. Its forward value is one, while the language-model loss can still update the router. A load-balancing loss with weight 0.01 prevents expert collapse, and a router z-loss with weight 0.001 controls logit magnitude. Near the end of training, the four average expert loads were approximately 31%, 24%, 21% and 24%.

2.3 Multi-token training and EMA

The main head predicts (x_{t+1}) . *Two training-only projections also predict (x_{t+2}) and (x_{t+3}) .* The loss is

$$L = L_{t+1} + 0.2L_{t+2} + 0.1L_{t+3} + 0.01L_{\text{balance}} + 0.001L_z.$$

The extra projections are initialized as identity matrices and share the tied vocabulary output matrix. They are not used by `predict_log_probs`, so they add no inference computation.

Training uses AdamW, BF16, batch size 32 and 12,000 updates. Each update processes 8,192 next-token targets. The peak learning rate is 0.0008 with 300 warm-up updates, AdamW betas `(0.9, 0.95)`, weight decay 0.1 and gradient clipping at 1.0. Training windows are shuffled in near non-overlapping 256-token epochs with a random offset.

An exponential moving average with decay 0.999 starts at update 2,000. Validation is run every 600 updates using the EMA parameters. The best neural checkpoint occurs at update 10,200 with 1.521357 validation BPB. The run continues to update 12,000, where validation BPB is 1.522353, so the earlier checkpoint is selected.

2.4 Exact causal suffix cache

For each prediction position, the suffix cache searches earlier positions in the current 256-token window for the longest exact suffix match. Earlier occurrences vote for the token that followed them. If (P_N) is the neural distribution and (P_C) is the continuation distribution,

$$P_S = (1 - \lambda_L)P_N + \lambda_L P_C,$$

where (L) is the match length. The cache weight rises from 0.05 for a one-token match to 0.55 for matches of eleven tokens or longer. A zero-length match receives zero weight. Only matches ending before the current position are allowed, and all cache state is reset between independent windows.

2.5 Training-derived 2- to 5-gram model

The final checkpoint contains an absolute-discount backoff model constructed from the supplied training tokens. The discount is 0.75. The bigram distribution is stored densely in FP16, while higher orders use sorted context keys, row offsets, target IDs, edge probabilities and backoff weights.

All trigram contexts are retained. Four-gram contexts must occur at least twice, and 5-gram contexts at least three times. Unreliable or unseen high-order contexts back off to the next lower order. The final probability is

$$P_{\text{final}} = 0.866P_S + 0.134P_{2-5\text{g}}.$$

The mixture is calculated in log space. No validation or test tokens are stored in the n-gram assets.

3. Results and ablations

3.1 Main results

PREDICTOR	VALIDATION BPB	IMPROVEMENT FROM PREVIOUS ROW
Supplied baseline, 4×128	2.071873	—
Dense modern GPT, 7×224	1.551581	0.520292
Sparse MoE + MTP	1.521357	0.030224
+ exact causal suffix cache	1.481449	0.039908
+ static trigram	1.466573	0.014876
+ pruned 4-gram	1.461338	0.005236
+ pruned 5-gram	1.460262	0.001076

The baseline-to-modern comparison includes increased model size and longer training, so it is not a single-mechanism ablation. The suffix and n-gram rows use the same neural checkpoint and isolate the incremental inference components. The suffix cache provides the largest individual gain, while the benefit of higher n-gram orders diminishes.

3.2 Backbone ablation

The modern components were compared with a matched legacy model using learned position embeddings, LayerNorm and GELU. Both models used seven blocks, width 224, dropout 0.1 and 78,643,200 processed targets.

BACKBONE	PARAMETERS	VALIDATION BPB
Legacy 7×224	4,745,440	1.561924
RMSNorm + RoPE + SwiGLU, 7×224	4,689,440	1.551581

The combined modern backbone improves BPB by 0.010343 at nearly equal parameter count. This experiment tests the three replacements together and does not assign a separate gain to each component.

3.3 MTP and depth ablations

ABLATION	CONTROLLED COMPARISON	RESULT
Remove MTP	Same 7-layer MoE at update 3,600	1.593282 without MTP vs approximately 1.5912 with MTP
Increase depth	Same MoE recipe at update 8,400	1.529232 for 8 layers vs 1.525147 for 7 layers

MTP gives a small but consistent early improvement. Increasing depth to eight layers improves early learning speed but gives worse later validation performance, so the seven-layer model is retained.

3.4 Evaluation resources

MEASUREMENT	RESULT	PROJECT LIMIT
Validation CPU FP32 time	35.28 s	≤5× baseline
Paired baseline CPU FP32 time	7.66 s	—
Relative CPU time	4.60×	≤5×

MEASUREMENT	RESULT	PROJECT LIMIT
Peak CPU working set	1.95 GiB	≤ 4 GiB
Inference assets	59.33 MiB	≤ 64 MiB
CPU validation BPB	1.460261659	—
CUDA validation BPB	1.460261671	—

The CPU and CUDA scores agree numerically. The method is close to both the CPU-time and asset-size limits, so additional active experts or model ensembling were not added.

3.5 Frozen full-test result

MODEL	FULL-TEST CPU FP32 BPB	SCORING TIME
Supplied baseline	2.101957	10.58 s in the earlier local run
Final submitted predictor	1.482848718	37.71 s

4. Reproduction

All commands are run from `code/`.

4.1 Environment and tests

```
conda create -n 7506_1 python=3.12 -y
conda activate 7506_1
python -m pip install torch==2.7.1 --index-url
https://download.pytorch.org/whl/cu126
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
```

4.2 Train the neural checkpoint

```
python train.py --implementation student `
--config configs/moe-mtp-original-7x224.json `
--device cuda --precision bf16 --threads 4 --seed 17 `
--steps 12000 --batch-size 32 --eval-every 600 `
--run-dir runs/moe-mtp-original-12000
```

This run processes 98,304,000 targets and took 1,341.55 training seconds on an RTX 4070 Laptop GPU. The selected source is `runs/moe-mtp-original-12000/best_checkpoint.pt`, saved at update 10,200.

4.3 Build the final checkpoint

```
python build_ngram_checkpoint.py `
  --source runs/moe-mtp-original-12000/best_checkpoint.pt `
  --output runs/moe-mtp-final/checkpoint.pt
```

The builder reads the supplied training split, creates the compact backoff assets and writes `lineage.json`. It does not perform additional gradient training.

4.4 Evaluate the frozen predictor

```
python evaluate.py `
  --checkpoint runs/moe-mtp-final/checkpoint.pt `
  --device cpu --precision fp32 --threads 4 `
  --split validation
```

After freezing the code, configuration and checkpoint:

```
python evaluate.py `
  --checkpoint runs/moe-mtp-final/checkpoint.pt `
  --device cpu --precision fp32 --threads 4 `
  --split test
```

The submitted score is the `bpb` field from the complete test JSON, not validation BPB or token perplexity.

5. Discussion

The final system combines predictors with complementary behaviour. The neural model learns distributed contextual representations, sparse experts increase conditional capacity, the suffix cache handles exact repetition inside the current window, and the backoff model supplies reliable local statistics from the training split. The suffix cache and trigram model account for most of the improvement after neural training.

The MoE improvement is not fully isolated from training length, EMA selection and the sampling schedule. The direct MTP ablation shows that MTP contributes only a small part of the early gain. Claims about the individual MoE contribution should therefore remain conservative.

The suffix weights, n-gram mixture weight and pruning thresholds were selected on one validation split. Repeated validation selection may introduce validation-specific tuning, although the test split remained unused during development. The final 5-gram gain is only 0.001076 BPB, indicating diminishing returns. Both CPU time and checkpoint size are also close to the project limits.

6. Conclusion

The selected predictor improves validation BPB from 2.071873 for the supplied baseline to 1.460262 while satisfying all measured resource limits. The modern backbone gives a controlled improvement over the matched legacy architecture. Sparse experts and multi-token training improve the neural checkpoint further, and the largest later gain comes from the causal suffix cache. Compact backoff n-grams provide an additional reduction without using external data or violating causal evaluation.

7. Reused work and AI assistance

The implementation is based on the course-provided GPT, training pipeline, tokenizer, data loader and evaluator. PyTorch's scaled dot-product attention and optimization primitives are used directly. No external training text or pretrained weights are used.

OpenAI Codex was used to help designing and implementing the experimental model, checking causality and resource limits, running and interpreting ablations and optimizing the sparse n-gram inference path.